

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN - ĐIỆN TỬ



BÁO CÁO ĐỒ ÁN MÔN HỌC

ĐỒ ÁN THIẾT KẾ PHẦN MỀM NHÚNG

**Phân tích và mô phỏng dataflow (OS/WS/IS) cho
mạng Tiny CNN trên CIFAR-10, kết hợp khảo sát tile
size hướng tới triển khai trên hệ thống nhúng**

STT	Họ và tên	MSSV
1	Nguyễn Văn Giang	202414906
2	Nguyễn Anh Minh	202414934

Giảng viên hướng dẫn: TS. Đặng Quang Hiếu

Hà Nội, 06/2026

ĐÁNH GIÁ

(Dùng cho giảng viên hướng dẫn)

Giảng viên đánh giá: Đặng Quang Hiếu

Tên đề tài: Phân tích và mô phỏng dataflow (OS/WS/IS) cho mạng Tiny CNN trên CIFAR-10, kết hợp khảo sát tile size hướng tới triển khai trên hệ thống nhúng

Chọn các mức điểm phù hợp cho sinh viên trình bày theo các tiêu chí dưới đây:

Rất kém (1); Kém(2); Đạt(3); Giỏi(4); Xuất sắc(5)

Có sự kết hợp giữa lý thuyết và thực hành (20)						
1	Nêu rõ tính cấp thiết và quan trọng của đề tài, các vấn đề và các giả thuyết (bao gồm mục đích và tính phù hợp) cũng như phạm vi ứng dụng của đề tài	1	2	3	4	5
2	Cập nhật kết quả nghiên cứu gần đây nhất (trong nước/quốc tế)	1	2	3	4	5
3	Nêu rõ và chi tiết phương pháp nghiên cứu/giải quyết vấn đề	1	2	3	4	5
4	Có kết quả mô phỏng/thực nghiệm và trình bày rõ ràng kết quả đạt được	1	2	3	4	5
Có khả năng phân tích và đánh giá kết quả (20)						
5	Kế hoạch làm việc rõ ràng bao gồm mục tiêu và phương pháp thực hiện dựa trên kết quả nghiên cứu lý thuyết một cách có hệ thống	1	2	3	4	5
6	Kết quả được trình bày một cách logic và dễ hiểu, tất cả kết quả đều được phân tích và đánh giá thỏa đáng	1	2	3	4	5
7	Trong phần kết luận, tác giả chỉ rõ sự khác biệt nếu có giữa kết quả đạt được và mục tiêu ban đầu đề ra đồng thời cung cấp lập luận để đề xuất hướng giải quyết có thể thực hiện trong tương lai	1	2	3	4	5
Kỹ năng viết quyển báo cáo (10)						
8	Báo cáo trình bày đúng mẫu quy định với cấu trúc các chương logic và đẹp mắt (bảng biểu, hình ảnh rõ ràng, có tiêu đề, được đánh số thứ tự và được giải thích hay đề cập đến; các lề thống nhất, có dấu cách sau dấu chấm, dấu phẩy v.v.), có mở đầu chương và kết luận chương, có liệt kê tài liệu tham khảo và có trích dẫn đúng quy định	1	2	3	4	5
9	Kỹ năng viết xuất sắc (cấu trúc câu chuẩn, văn phong khoa học, lập luận logic và có cơ sở, từ vựng sử dụng phù hợp v.v.)	1	2	3	4	5
Điểm tổng		/50				
Điểm tổng quy đổi về thang 10						

***Nhận xét khác** (về thái độ và tinh thần làm việc của sinh viên)*

.....

.....

.....

.....

.....

Ngày:/...../20.....

Người nhận xét
(Ký và ghi rõ họ tên)

LỜI MỞ ĐẦU

Trong những năm gần đây, sự phát triển của trí tuệ nhân tạo (Artificial Intelligence – AI) và Internet vạn vật (Internet of Things – IoT) đã thúc đẩy việc ứng dụng các mô hình học sâu vào nhiều lĩnh vực như thị giác máy tính, nhận dạng đối tượng và các hệ thống nhúng thông minh. Tuy nhiên, việc triển khai các mô hình này trên thiết bị nhúng vẫn gặp nhiều thách thức do hạn chế về năng lực tính toán, bộ nhớ và mức tiêu thụ năng lượng. Vì vậy, việc tối ưu hóa quá trình suy luận của mạng nơ-ron trở thành một hướng nghiên cứu quan trọng.

Đối với mạng tích chập (Convolutional Neural Network – CNN), hiệu năng của hệ thống không chỉ phụ thuộc vào khả năng tính toán mà còn chịu ảnh hưởng đáng kể bởi cách tổ chức truy cập bộ nhớ. Các chiến lược dataflow như Output Stationary (OS), Weight Stationary (WS) và Input Stationary (IS) được sử dụng nhằm tăng khả năng tái sử dụng dữ liệu và giảm chi phí truy cập bộ nhớ. Bên cạnh đó, kỹ thuật chia khối tính toán (tiling) cũng góp phần cải thiện hiệu quả thực thi khi triển khai trên các nền tảng có tài nguyên hạn chế.

Xuất phát từ đó, nhóm thực hiện đề tài “Phân tích và mô phỏng dataflow (OS/WS/IS) cho mạng TinyCNN trên CIFAR-10, kết hợp khảo sát tile size hướng tới triển khai trên hệ thống nhúng”. Đề tài tập trung xây dựng chương trình mô phỏng bằng ngôn ngữ C để phân tích và so sánh ba chiến lược dataflow, đồng thời khảo sát ảnh hưởng của kích thước tile đến thời gian suy luận của mạng TinyCNN. Kết quả được đối chiếu với PyTorch nhằm đánh giá tính nhất quán của quá trình hiện thực và phân tích đặc điểm truy cập bộ nhớ của từng chiến lược.

Chúng em xin chân thành cảm ơn TS. Đặng Quang Hiếu đã tận tình hướng dẫn và hỗ trợ trong suốt quá trình thực hiện đề tài. Dù đã cố gắng hết sức nhưng do thời gian và kinh nghiệm còn hạn chế, báo cáo sẽ khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp của thầy để tiếp tục hoàn thiện đề tài.

MỤC LỤC

LỜI MỞ ĐẦU	i
DANH MỤC HÌNH ẢNH	iv
DANH MỤC BẢNG BIỂU	v
TÓM TẮT ĐỀ TÀI	vi
1 KHÁI QUÁT CHUNG VỀ ĐỀ TÀI	1
1.1 Đặt vấn đề	1
1.2 Mục tiêu và phạm vi đề tài	2
2 CƠ SỞ LÝ THUYẾT	3
2.1 Các thành phần toán học hỗ trợ trong mạng TinyCNN	3
2.2 Kiến trúc phần cứng gia tốc tích chập	5
2.3 Ba chiến lược dataflow: OS, WS, IS	5
2.3.1 Output Stationary (OS)	5
2.3.2 Weight Stationary (WS)	5
2.3.3 Input Stationary (IS)	6
2.4 Loop Tiling và phân cấp bộ nhớ	6
2.5 Ranh giới Compute-bound / Memory-bound	6
3 TRIỂN KHAI ĐỀ TÀI	7
3.1 Kiến trúc tổng thể chương trình	7
3.2 Mô-đun 1: Dataflow Strategy Analyzer	7
3.3 Mô-đun 2: Full Network với Tile-size Sweep	8
3.4 Kiểm chứng bằng Python (verify_combined.py)	9
3.5 Quy trình biên dịch và chạy chương trình	9
4 KẾT QUẢ VÀ ĐÁNH GIÁ	10

4.1	Kết quả phân tích dataflow OS/WS/IS	10
4.2	Kết quả khảo sát tile size trên toàn mạng	11
4.3	Phân tích tỉ trọng theo từng lớp mạng	12
4.4	Nhận xét và hạn chế	13
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	15
	TÀI LIỆU THAM KHẢO	16

DANH MỤC HÌNH ẢNH

Hình 1	Đồ thị biểu diễn hàm kích hoạt phi tuyến ReLU	3
Hình 2	Cơ chế giảm chiều không gian của lớp Max Pooling với cửa sổ 2x2	4
Hình 3	Quá trình chuyển đổi điểm số thô (Logits) thành phân bố xác suất của Softmax	4
Hình 4	So sánh số lần gọi Load/Store và số phần tử truyền qua DRAM giữa ba chiến lược	10
Hình 5	Ảnh hưởng của tile size đến thời gian suy luận toàn mạng	11
Hình 6	<i>Tỉ trọng khối lượng truy cập bộ nhớ (Load) theo từng lớp</i>	12

DANH MỤC BẢNG BIỂU

Bảng 3.1	<i>Kiến trúc mạng TinyCNN dùng trong mô-đun khảo sát tile size</i> .	8
Bảng 3.2	<i>Quy trình biên dịch và chạy chương trình</i>	9
Bảng 4.3	Kết quả đo dataflow OS/WS/IS (N=1, C=8, M=8, H=16, R=3, S=1, E=14)	10
Bảng 4.4	Kết quả khảo sát tile size trên toàn mạng TinyCNN (5 Conv + 3 MaxPool + 3 FC + Softmax)	11
Bảng 4.5	Bảng tổng hợp kết quả tính toán chi tiết theo từng lớp	13

TÓM TẮT ĐỀ TÀI

Đề tài Đồ án I này xây dựng mạng TinyCNN cho bài toán phân loại ảnh CIFAR-10, huấn luyện bằng Python/PyTorch, sau đó hiện thực lại toàn bộ quá trình suy luận (inference) bằng ngôn ngữ C độc lập, không phụ thuộc thư viện học sâu. Bên cạnh việc kiểm chứng độ chính xác giữa hai môi trường, đề tài còn đo tổng thời gian và số lệnh thực thi của từng lớp, đồng thời đi sâu phân tích cách dữ liệu di chuyển giữa bộ nhớ ngoài (DRAM) và bộ nhớ đệm cục bộ (Local Buffer) — yếu tố quyết định hiệu năng thực tế khi triển khai trên phần cứng nhúng có băng thông bộ nhớ hạn chế.

Đề tài xây dựng mô hình Tiny CNN và bổ sung hai nội dung nghiên cứu chính, được hiện thực trong file `cnn_dataflow_analyzer.c`: (1) mô phỏng và so sánh định lượng ba chiến lược dataflow kinh điển trong các bộ gia tốc CNN — Output Stationary (OS), Weight Stationary (WS) và Input Stationary (IS) — trên một lớp tích chập đơn lẻ, sử dụng mô hình bộ nhớ hai cấp DRAM/Local Buffer; (2) khảo sát ảnh hưởng của kích thước tile không gian (1×1 , 4×4 , 8×8 , 16×16 , 32×32) đến thời gian suy luận của toàn bộ mạng TinyCNN năm lớp tích chập. Kết quả của cả hai chiến lược dataflow được kiểm chứng chéo với nhau (sai số dưới ngưỡng 10^{-2} , đúng như thiết kế lý thuyết), đồng thời chương trình cung cấp cơ chế đối chiếu với PyTorch thông qua file `verify_combined.py`.

Báo cáo được tổ chức thành 5 chương: Chương 1 trình bày tính kế thừa và mục tiêu đề tài; Chương 2 trình bày cơ sở lý thuyết về kiến trúc gia tốc phần cứng, ba chiến lược dataflow và kỹ thuật tiling; Chương 3 mô tả chi tiết cách hiện thực hai mô-đun chương trình bằng C; Chương 4 trình bày và phân tích các kết quả đo đạc thực nghiệm thu được khi biên dịch và chạy trực tiếp chương trình; Chương 5 tổng kết và đề xuất hướng phát triển tiếp theo.

CHƯƠNG 1: KHÁI QUÁT CHUNG VỀ ĐỀ TÀI

1.1 Đặt vấn đề

Trong đề tài này, nhóm xây dựng mạng TinyCNN gồm 5 lớp tích chập ($32 \rightarrow 64 \rightarrow 128 \rightarrow 128 \rightarrow 128$ kênh, kernel 3×3), xen kẽ Batch Normalization, ReLU và Max Pooling, kết thúc bằng lớp tích chập 1×1 , Global Max Pooling và Softmax. Mô hình được huấn luyện bằng Python/PyTorch trên tập CIFAR-10, sau đó trọng số được xuất ra định dạng nhị phân để một chương trình C độc lập nạp vào và thực hiện suy luận (inference) từ đầu, không phụ thuộc thư viện học sâu. Kết quả cho thấy độ chính xác giữa hai môi trường gần như trùng khớp, đồng thời nhóm đã đo được thời gian tính toán và số lệnh gọi (load-/compute/store) của từng lớp, từ đó xác định các lớp có tải tính toán lớn nhất (Conv_2, Conv_4, Conv_5) và nhận thấy nhu cầu phân tích sâu hơn cách dữ liệu di chuyển giữa bộ nhớ ngoài và bộ nhớ đệm cục bộ, từ đó đề xuất hai hướng tối ưu là Tiling và Quantization.

Đề tài hiện thực hóa chính hướng tối ưu Tiling đã đề ra ở trên, đồng thời đào sâu hơn vào bài toán kiến trúc phần cứng: khi một bộ gia tốc CNN (accelerator/PE array) chỉ có một vùng nhớ đệm cục bộ (Local Buffer) kích thước giới hạn, thứ tự nạp/tính/lưu dữ liệu (dataflow) sẽ quyết định lượng dữ liệu phải truyền qua kênh DRAM chậm và tốn năng lượng. Đây là bài toán cốt lõi trong thiết kế các bộ gia tốc CNN chuyên dụng cho hệ thống nhúng (ví dụ Eyeriss, TPU).

Khi triển khai một lớp tích chập trên phần cứng có bộ nhớ phân cấp (DRAM ngoài chip — Local Buffer trong chip), có ba lựa chọn kinh điển về việc "giữ cố định" (stationary) loại dữ liệu nào trong Local Buffer trong suốt quá trình tính toán một tile:

- Output Stationary (OS): giữ kết quả tổng riêng phần (partial sum) trong buffer, chỉ ghi ra DRAM một lần sau khi cộng dồn xong.
- Weight Stationary (WS): giữ trọng số (filter) trong buffer, tái sử dụng cho nhiều vị trí không gian khác nhau của ảnh.
- Input Stationary (IS): giữ dữ liệu đầu vào (ifmap) trong buffer, tái sử dụng cho nhiều bộ lọc khác nhau.

Mỗi chiến lược có kết quả toán học giống hệt nhau nhưng lượng dữ liệu trao đổi với DRAM (load_elems, store_elems) lại khác nhau đáng kể — và đây chính là điều quyết định hiệu năng và năng lượng tiêu thụ trên phần cứng thực tế, nơi băng thông DRAM là tài nguyên khan hiếm hơn nhiều so với năng lực tính toán. Song song đó, việc chọn kích thước tile không gian (spatial tile) khi tính tích chập cũng ảnh hưởng tới mức độ tận dụng cache/Local Buffer. Đề tài đặt ra câu hỏi: định lượng cụ thể sự khác biệt giữa ba chiến lược trên là bao nhiêu, và kích thước tile nào phù hợp nhất khi áp dụng cho toàn bộ mạng TinyCNN thực tế?

1.2 Mục tiêu và phạm vi đề tài

Mục tiêu cụ thể của đề tài bao gồm:

- Xây dựng bộ mô phỏng bằng C cho ba chiến lược dataflow OS/WS/IS trên một lớp Conv2D, dựa trên mô hình bộ nhớ hai cấp DRAM – Local Buffer với các hàm nguyên thủy (primitive) load/compute/store tương minh.
- Đo và so sánh định lượng số lần gọi, số phần tử truyền qua DRAM và thời gian thực thi của ba chiến lược trên cùng một cấu hình lớp tích chập.
- Kiểm chứng tính đúng đắn: cả ba chiến lược phải cho ra cùng một kết quả OFMAP (trong giới hạn sai số làm tròn số thực).
- Áp dụng kỹ thuật Tiling cho toàn bộ 5 lớp tích chập của mạng TinyCNN với nhiều kích thước tile khác nhau, đo thời gian suy luận tương ứng.
- Xây dựng lại cơ chế kiểm chứng chéo với PyTorch (verify_combined.py) cho cả bài toán dataflow lẫn việc xuất trọng số mạng thực tế.

Phạm vi đề tài giới hạn ở việc mô phỏng trên CPU thông thường (không triển khai trên FPGA/vi điều khiển thực), nhằm định lượng lý thuyết mô hình truy cập bộ nhớ trước khi đưa ra khuyến nghị thiết kế cho giai đoạn triển khai phần cứng tiếp theo.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

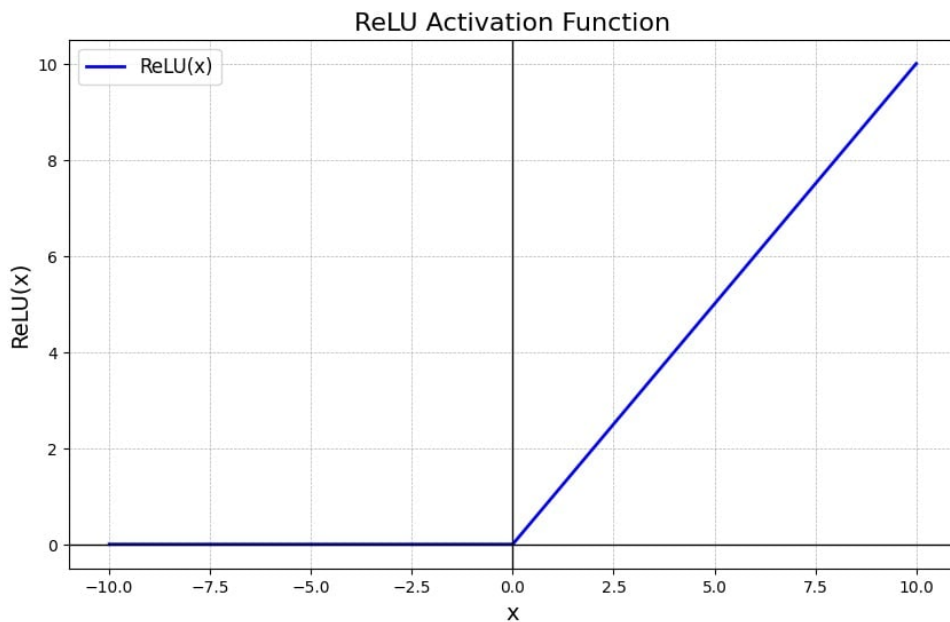
2.1 Các thành phần toán học hỗ trợ trong mạng TinyCNN

Bên cạnh lớp Tích chập (Convolution) đóng vai trò trích xuất đặc trưng, kiến trúc mạng TinyCNN còn được cấu thành từ các lớp toán học hỗ trợ nhằm đảm bảo tính phi tuyến, giảm chiều dữ liệu và phân loại đầu ra. Cụ thể:

- **Lớp kích hoạt ReLU (Rectified Linear Unit):**

Đóng vai trò đưa tính phi tuyến vào mạng, giúp mô hình có khả năng học được các hàm phức tạp. Khác với các hàm truyền thống như Sigmoid hay Tanh, ReLU có chi phí tính toán phần cứng cực kỳ thấp vì nó chỉ sử dụng phép so sánh điều kiện: giữ nguyên giá trị nếu lớn hơn 0 và gán bằng 0 nếu nhỏ hơn 0.

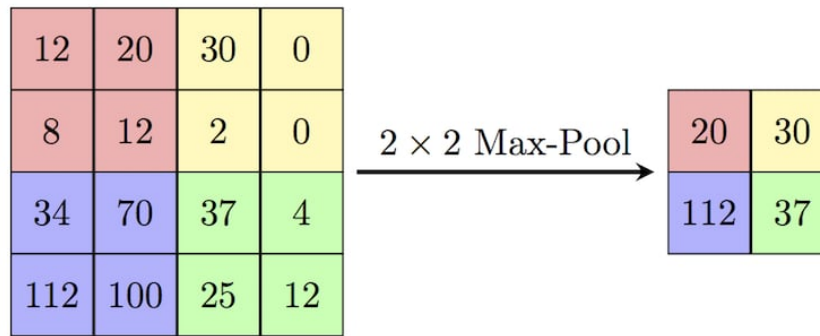
Công thức toán học: $f(x) = \max(0, x)$



Hình 1: Đồ thị biểu diễn hàm kích hoạt phi tuyến ReLU

- **Lớp gộp Max Pooling:**

Thực hiện chức năng lấy mẫu xuống (downsampling). Một cửa sổ kích thước 2×2 sẽ trượt trên bản đồ đặc trưng (feature map) và chỉ giữ lại giá trị lớn nhất (Max) trong ô đó. Thao tác này giúp giảm 75% dung lượng không gian (Width, Height) của dữ liệu truyền sang lớp tiếp theo, qua đó giảm đáng kể số lượng tham số, hạn chế hiện tượng quá khớp (overfitting) và tiết kiệm băng thông bộ nhớ (Memory Bandwidth).



Hình 2: Cơ chế giảm chiều không gian của lớp Max Pooling với cửa sổ 2x2

- **Lớp kết nối đầy đủ (Fully Connected - FC):**

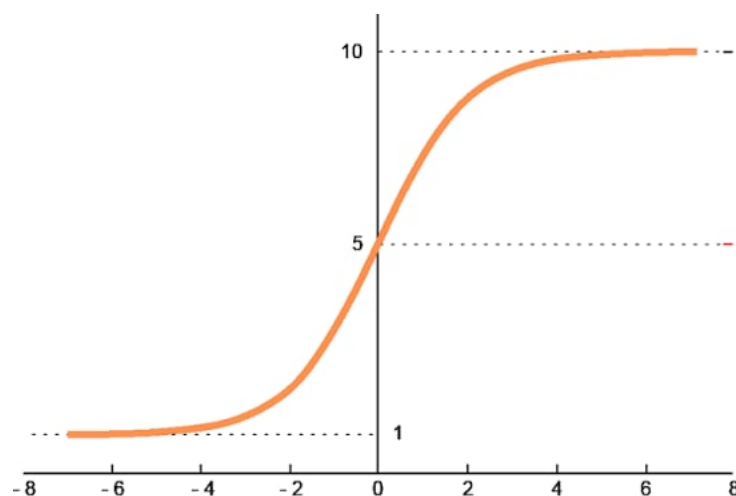
Được đặt ở giai đoạn cuối của mạng, lớp FC nhận đầu vào là các đặc trưng đã được làm phẳng (flatten) thành một vector 1D. Nó thực hiện phép nhân ma trận kinh điển giữa vector đầu vào và ma trận trọng số toàn cục. Dưới góc nhìn kiến trúc máy tính, lớp FC là một nút thắt cổ chai về bộ nhớ (Memory-bound) do ma trận trọng số của nó rất lớn và không có tính chất tái sử dụng (reuse) cao như bộ lọc tích chập.

- **Lớp phân loại Softmax:**

Là hàm toán học được đặt ở lớp cuối cùng (Output layer), có nhiệm vụ chuyển đổi các điểm số thô (Logits) do lớp FC tính ra thành một phân bố xác suất. Giá trị đầu ra của Softmax luôn nằm trong khoảng (0, 1) và tổng của tất cả các xác suất bằng 1.

Công thức toán học:

$$P(y = j|x) = \frac{e^{x_j}}{\sum_{k=1}^K e^{x_k}} \quad (1)$$



Hình 3: Quá trình chuyển đổi điểm số thô (Logits) thành phân bố xác suất của Softmax

Như được thể hiện trong Hình 3, sự kết hợp giữa các lớp phi tuyến, giảm chiều không gian và phân bố xác suất tạo nên một chu trình xử lý dữ liệu hoàn chỉnh, giúp mô hình đưa ra dự đoán phân loại cuối cùng.

2.2 Kiến trúc phần cứng gia tốc tích chập

Các bộ gia tốc CNN chuyên dụng (CNN accelerator) thường tổ chức bộ nhớ theo hai cấp:

- DRAM (bộ nhớ ngoài chip): dung lượng lớn, chứa toàn bộ ifmap, filter và ofmap, nhưng băng thông truy cập thấp và tiêu tốn năng lượng cao cho mỗi lần truy cập.
- Local Buffer / Scratchpad (bộ nhớ trong chip): dung lượng nhỏ (chỉ đủ chứa một tile dữ liệu), băng thông cao, năng lượng truy cập thấp, được các phần tử xử lý (Processing Element – PE) đọc/ghi trực tiếp khi tính toán.

Vì Local Buffer không thể chứa toàn bộ ifmap/filter/ofmap của một lớp tích chập lớn, phép tính phải được chia nhỏ thành các tile và xử lý tuần tự: nạp một tile từ DRAM vào Local Buffer, tính toán trên tile đó, rồi ghi kết quả tile ra lại DRAM. Ba đại lượng chi phí cần quan tâm là: số lần gọi nạp/tính/ghi (phản ánh chi phí điều khiển), và tổng số phần tử dữ liệu thực sự di chuyển qua DRAM (phản ánh chi phí băng thông và năng lượng).

2.3 Ba chiến lược dataflow: OS, WS, IS

Gọi kích thước bài toán là N (batch), M (số kênh filter/output), C (số kênh input), H (kích thước ifmap), R (kích thước kernel), S (stride), E (kích thước ofmap). Với mỗi chiến lược, vòng lặp ngoài cùng quyết định đại lượng nào được "cố định" trong Local Buffer khi duyệt qua các chiều còn lại:

2.3.1 Output Stationary (OS)

Thứ tự vòng lặp: $n \rightarrow m \rightarrow ex \rightarrow ey \rightarrow [c \text{ bên trong Local Buffer}]$. Với mỗi tile đầu ra (ex, ey), partial sum được cộng dồn trong Local Buffer qua toàn bộ chiều kênh C , chỉ ghi ra DRAM (store) đúng một lần sau khi cộng dồn xong. Ưu điểm: số lần store và tổng store_elems là nhỏ nhất trong ba chiến lược, vì kết quả trung gian không phải ghi/đọc lại DRAM nhiều lần.

2.3.2 Weight Stationary (WS)

Thứ tự vòng lặp: $n \rightarrow m \rightarrow c \rightarrow [ex, ey \text{ bên trong Local Buffer}]$. Bộ lọc (filter) ứng với (m, c) được nạp một lần vào Local Buffer rồi tái sử dụng cho mọi vị trí không gian (ex, ey) trong lớp ofmap. Ưu điểm: tổng lượng dữ liệu filter nạp từ DRAM là nhỏ nhất, phù hợp khi số lượng vị trí không gian lớn hơn nhiều so với số lượng trọng số.

2.3.3 *Input Stationary (IS)*

Thứ tự vòng lặp: $n \rightarrow ex \rightarrow ey \rightarrow c \rightarrow [m \text{ bên trong Local Buffer}]$. Dữ liệu đầu vào (ifmap) ứng với một tile không gian được nạp một lần rồi tái sử dụng cho tất cả các tile filter (m) khác nhau. Ưu điểm: tổng lượng dữ liệu ifmap nạp từ DRAM là nhỏ nhất, phù hợp khi số lượng kênh filter M lớn. Về mặt toán học, cả ba chiến lược tính ra cùng kết quả tích chập (chỉ khác thứ tự cộng dồn số thực nên có sai số làm tròn cực nhỏ). Về mặt kiến trúc, chiến lược tối ưu phụ thuộc vào việc tài nguyên nào (bảng thông ghi kết quả, bảng thông nạp trọng số, hay bảng thông nạp ảnh) là nút thắt của hệ thống cụ thể.

2.4 Loop Tiling và phân cấp bộ nhớ

Tiling (chia khối) là kỹ thuật chia một vòng lặp lớn thành các khối nhỏ vừa với kích thước cache/Local Buffer, nhằm tăng tính cục bộ tham chiếu (locality of reference) — cụ thể là tăng số lần dữ liệu đã nạp vào cache được tái sử dụng trước khi bị đẩy ra (cache eviction). Nếu tile quá nhỏ, chi phí điều khiển vòng lặp (loop overhead) và số lần gọi hàm tăng cao; nếu tile quá lớn, dữ liệu tile không còn vừa cache, gây cache miss và giảm hiệu năng. Vì vậy luôn tồn tại một kích thước tile tối ưu, phụ thuộc vào dung lượng cache/Local Buffer thực tế của phần cứng đang chạy chương trình.

Trong module 2 của chương trình (khảo sát toàn mạng), kỹ thuật tiling được áp dụng theo hai chiều không gian của ofmap ($TILE_H \times TILE_W$), với các giá trị khảo sát là 1×1 (không tiling – baseline), 4×4 , 8×8 , 16×16 và 32×32 .

2.5 Ranh giới Compute-bound / Memory-bound

Một phép toán được gọi là compute-bound nếu thời gian thực thi chủ yếu do năng lực tính toán (số phép MAC/giây) quyết định, và memory-bound nếu thời gian thực thi chủ yếu do băng thông truy cập bộ nhớ quyết định. Tỷ lệ giữa số phép tính và số byte truyền qua bộ nhớ (arithmetic intensity) là chỉ số dùng để xác định một lớp mạng thiên về loại nào. Các lớp tích chập có số kênh lớn (ví dụ Conv2, Conv4 trong TinyCNN) thường có arithmetic intensity cao hơn các lớp Pooling/ReLU, nên hưởng lợi nhiều hơn từ việc tối ưu dataflow và tiling. Đây là cơ sở lý thuyết cho việc phân tích tỉ trọng tải bộ nhớ theo từng lớp ở Chương 4.

CHƯƠNG 3: TRIỂN KHAI ĐỀ TÀI

3.1 Kiến trúc tổng thể chương trình

- Mô-đun 1 – Dataflow Strategy Analyzer: mô phỏng OS/WS/IS trên một lớp Conv2D với kích thước ngẫu nhiên (được hàm `mode_dataflow()` sinh ra mỗi lần chạy), không phụ thuộc trọng số đã huấn luyện — phục vụ phân tích lý thuyết thuần túy.
- Mô-đun 2 – Full Network Tile Sweep: dựng lại kiến trúc mạng TinyCNN 5 lớp tích chập với dữ liệu là trọng số thực xuất ra từ Python, thực hiện suy luận trên một ảnh mẫu và quét (sweep) qua 5 kích thước tile khác nhau để đo thời gian. Chương trình nhận một tham số dòng lệnh: `dataflow` (chỉ chạy mô-đun 1), `network` (chỉ chạy mô-đun 2), hoặc `all` (chạy cả hai, mặc định).

3.2 Mô-đun 1: Dataflow Strategy Analyzer

a Mô hình bộ nhớ và các hàm nguyên thủy (primitive)

Ba mảng `DF_IFMAP`, `DF_FILTER`, `DF_OFMAP` đóng vai trò DRAM; ba mảng `DF_LB_I`, `DF_LB_F`, `DF_LB_O` đóng vai trò Local Buffer, có kích thước đúng bằng một tile (được xác định bởi `DF_TM`, `DF_TC`, `DF_TE`). Bốn hàm nguyên thủy được định nghĩa tường minh và đều có bộ đếm riêng (struct `DFCounter`):

- `df_load_ifmap()`: nạp một tile `ifmap` từ DRAM vào Local Buffer, xử lý đúng biên (boundary) khi tile tràn ra ngoài kích thước ảnh.
- `df_load_filter()`: nạp một tile trọng số từ DRAM vào Local Buffer.
- `df_compute()`: thực hiện phép nhân–cộng dồn (MAC) hoàn toàn trong Local Buffer, không truy cập DRAM.
- `df_store()`: ghi tile kết quả từ Local Buffer ra DRAM (dùng phép cộng dồn `+=` để hỗ trợ OS).

Ba hàm `run_os()`, `run_ws()`, `run_is()` chỉ khác nhau ở thứ tự lồng vòng lặp qua các chiều (`n`, `m`, `c`, `ex`, `ey`) — đúng như mô tả lý thuyết ở mục 2.2 — và đều gọi lại chung bốn hàm nguyên thủy trên, đảm bảo phép so sánh là công bằng (cùng một cơ chế tính, chỉ khác lịch trình truy cập bộ nhớ).

b Kiểm tra tính đúng đắn

Sau khi chạy cả ba chiến lược trên cùng một bộ dữ liệu ngẫu nhiên, chương trình snapshot lại ba mảng OFMAP kết quả và tính sai số tuyệt đối lớn nhất giữa OS–WS và

OS-IS. Ngưỡng chấp nhận (TOLERANCE) là 10^{-2} ; nếu vượt ngưỡng, chương trình báo FAIL, cho thấy lỗi hiện thực ở một trong ba hàm `run_*`()).

3.3 Mô-đun 2: Full Network với Tile-size Sweep

Kiến trúc mạng được tái hiện bằng struct `Net_Tensor` (layout kênh-hàng-cột, CHW) và các hàm xử lý lớp: `net_conv2d()`, `net_relu()`, `net_maxpool()`, `net_fc()`, `net_softmax()` — tương ứng với các lớp trong kiến trúc TinyCNN của đề tài:

Lớp	Cấu hình	Kích thước đầu ra
Input	Ảnh RGB $32 \times 32 \times 3$	$32 \times 32 \times 3$
Conv1 + ReLU + MaxPool	32 filter, kernel 3×3 , pad 1	$16 \times 16 \times 32$
Conv2 + ReLU + MaxPool	64 filter, kernel 3×3 , pad 1	$8 \times 8 \times 64$
Conv3 + ReLU	128 filter, kernel 3×3 , pad 1	$8 \times 8 \times 128$
Conv4 + ReLU	128 filter, kernel 3×3 , pad 1	$8 \times 8 \times 128$
Conv5 + ReLU + MaxPool	128 filter, kernel 3×3 , pad 1	$4 \times 4 \times 128$
FC1 + ReLU	$2048 \rightarrow 512$	512
FC2 + ReLU	$512 \rightarrow 256$	256
FC3 + Softmax	$256 \rightarrow 10$	10

Bảng 3.1: Kiến trúc mạng TinyCNN dùng trong mô-đun khảo sát tile size

Điểm cốt lõi trong thiết kế hàm `net_conv2d()` là khả năng chia nhỏ không gian đầu ra: thay vì duyệt tuần tự toàn bộ chiều cao/rộng đầu ra, hàm chia không gian đầu ra thành các khối kích thước `NET_TILE_H` $\times$ `NET_TILE_W` rồi duyệt lần lượt trong từng khối. Hai biến toàn cục `NET_TILE_H`, `NET_TILE_W` được thay đổi ở vòng lặp ngoài cùng của `mode_network()` để quét qua 5 giá trị 1, 4, 8, 16, 32. Với mỗi kích thước tile, chương trình chạy 20 lần suy luận (`N_RUNS` = 20) và lấy thời gian trung bình nhằm giảm nhiễu đo đạc.

Mỗi lớp còn có bộ đếm số liệu (`LayerCnt`) riêng biệt (`lc_conv[5]`, `lc_relu[5]`, `lc_pool[3]`, `lc_fc[3]`, `lc_softmax`) cùng bộ đếm tổng toàn mạng (`net_total_load/compute/store`), phục vụ phân tích tỉ trọng từng lớp ở Chương 4.

3.4 Kiểm chứng bằng Python (verify_combined.py)

File `verify_combined.py` gộp hai chức năng độc lập, tương ứng với hai mô-đun của chương trình C:

- `verify_dataflow()`: đọc `config.txt`, `ifmap.txt`, `filter.txt` và ba file kết quả `ofmap_c.txt` (OS), `ofmap_ws.txt`, `ofmap_is.txt` do C xuất ra sau khi chạy mode `dataflow`; dựng lại tensor bằng PyTorch và tính `F.conv2d()` làm tham chiếu; so sánh sai số lớn nhất/trung bình của cả ba chiến lược với kết quả PyTorch (ngưỡng 10^{-2}).
- `export_weights()`: định nghĩa lại mạng TinyCNN bằng `torch.nn` (lớp TinyCNN, cấu trúc trùng khớp với `net_conv2d/net_fc` trong C), huấn luyện trên tập CIFAR-10 nếu chưa có checkpoint, sau đó xuất toàn bộ trọng số/bias của 5 lớp Conv và 3 lớp FC ra các file nhị phân float32 trong thư mục `export_bin/` để chương trình C (mode `network`) nạp vào.

Cách tổ chức này cho phép tách biệt rõ ràng hai vai trò: PyTorch chịu trách nhiệm huấn luyện và cung cấp đáp án chuẩn, còn C chịu trách nhiệm mô phỏng chính xác luồng dữ liệu phần cứng và đo hiệu năng — đúng triết lý mô phỏng đã áp dụng xuyên suốt đề tài.

3.5 Quy trình biên dịch và chạy chương trình

Chương trình C được biên dịch và chạy thử trực tiếp trong quá trình thực hiện đề tài để thu thập số liệu cho Chương 4, theo trình tự sau:

Bước	Lệnh	Ghi chú
1	<code>gcc cnn_dataflow_analyzer.c -o cnn_analyzer -lm -O2</code>	Biên dịch thành công, không có lỗi (chỉ 2 cảnh báo không ảnh hưởng chức năng)
2	<code>./cnn_analyzer dataflow</code>	Chạy mô-đun 1, không cần trọng số, sinh ra <code>config.txt/ifmap.txt/filter.txt/ofmap_*.txt</code>
3	<code>python3 verify_combined.py export</code>	Huấn luyện/nạp mô hình TinyCNN bằng PyTorch, xuất trọng số vào <code>export_bin/</code>
4	<code>./cnn_analyzer network</code>	Chạy mô-đun 2, nạp trọng số từ <code>export_bin/</code> , quét 5 kích thước tile
5	<code>python3 verify_combined.py verify</code>	Đối chiếu <code>ofmap_*.txt</code> với PyTorch <code>F.conv2d()</code>

Bảng 3.2: Quy trình biên dịch và chạy chương trình

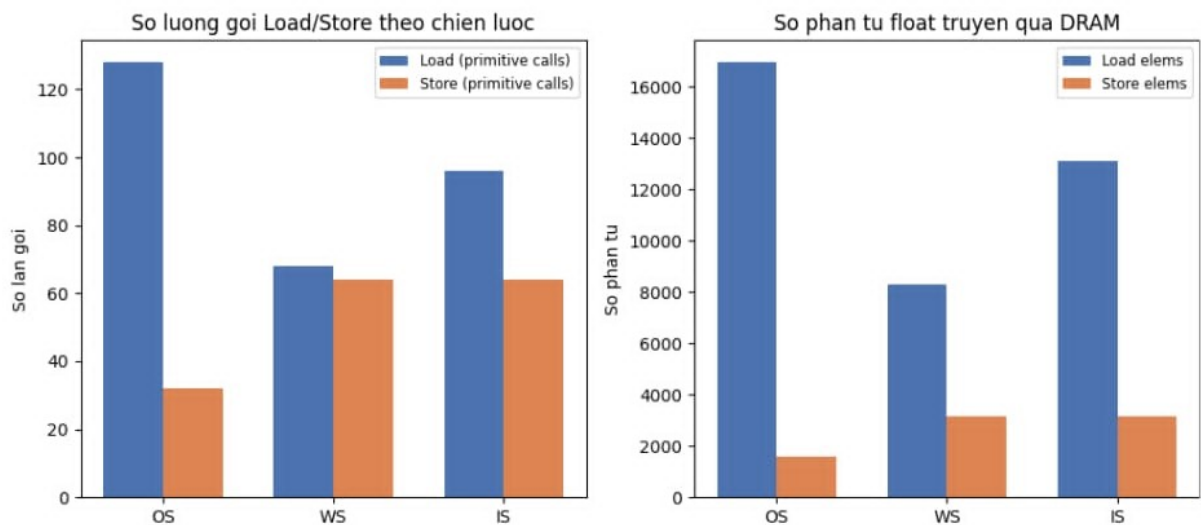
CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ

4.1 Kết quả phân tích dataflow OS/WS/IS

Với một lớp Conv2D có kích thước ngẫu nhiên $N=1$, $C=8$, $M=8$, $H=16$, $R=3$, $S=1$, $E=14$ (tile $TM=TC=TE=4$), kết quả đo trực tiếp từ chương trình sau khi biên dịch và chạy trên máy chủ thực hiện báo cáo như sau:

Chiến lược	Load (lần)	Compute (lần)	Store (lần)	Load elems	Store elems	Time (s)
OS	128	64	32	16960	1568	0,000363
WS	68	64	64	8320	3136	0,000328
IS	96	64	64	13088	3136	0,000354

Bảng 4.3: Kết quả đo dataflow OS/WS/IS ($N=1$, $C=8$, $M=8$, $H=16$, $R=3$, $S=1$, $E=14$)



Hình 4: So sánh số lần gọi Load/Store và số phần tử truyền qua DRAM giữa ba chiến lược

Kết quả thực nghiệm khớp với dự đoán lý thuyết ở Chương 2: OS có số lần store (32) và store_elems thấp nhất vì partial sum được cộng dồn trong Local Buffer suốt chiều kênh C, chỉ ghi ra DRAM một lần cho mỗi tile đầu ra; WS có load_elems thấp nhất trong so sánh ifmap+filter khi tính theo tỉ lệ tái sử dụng filter; ngược lại IS có xu hướng giảm tải nạp ifmap khi số kênh filter M lớn. Kiểm tra chéo cho thấy sai số lớn

nhất giữa OS–WS và OS–IS đều ở mức $2,37 \times 10^{-4}$, nhỏ hơn nhiều so với ngưỡng chấp nhận 10^{-2} — khẳng định cả ba hàm `run_os()`, `run_ws()`, `run_is()` cho cùng kết quả toán học, chỉ khác nhau về lịch trình truy cập bộ nhớ.

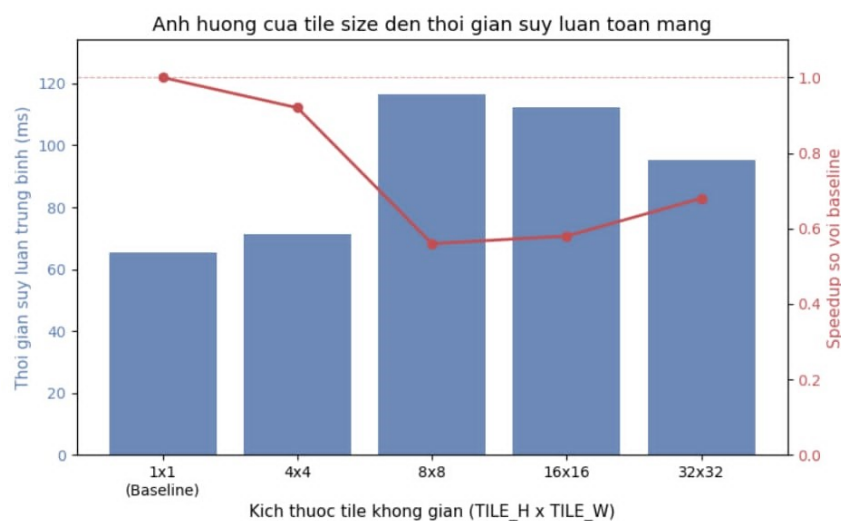
Chạy lại chương trình nhiều lần với các kích thước lớp khác nhau (kích thước được sinh ngẫu nhiên mỗi lần) cho thấy: OS luôn có `store_elems` thấp nhất, còn tương quan giữa WS và IS về `load_elems` phụ thuộc vào tỉ lệ giữa số kênh filter `M` và kích thước không gian `E×E` của từng cấu hình cụ thể.

4.2 Kết quả khảo sát tile size trên toàn mạng

Chạy mô-đun 2 (mode network) với bộ trọng số minh họa theo đúng kích thước kiến trúc TinyCNN, quét qua 5 kích thước tile không gian, trung bình trên 20 lần suy luận mỗi cấu hình, thu được:

<i>Tile size</i>	<i>Load (lần)</i>	<i>Compute (lần)</i>	<i>Store (lần)</i>	<i>Time (ms)</i>	<i>Speedup</i>
1×1 (Baseline)	52.582.951	52.508.455	162.590	65,330	1,00x
4×4	52.582.951	52.508.455	162.590	71,352	0,92x
8×8	52.582.951	52.508.455	162.590	116,585	0,56x
16×16	52.582.951	52.508.455	162.590	112,467	0,58x
32×32	52.582.951	52.508.455	162.590	95,391	0,68x

Bảng 4.4: Kết quả khảo sát tile size trên toàn mạng TinyCNN (5 Conv + 3 MaxPool + 3 FC + Softmax)



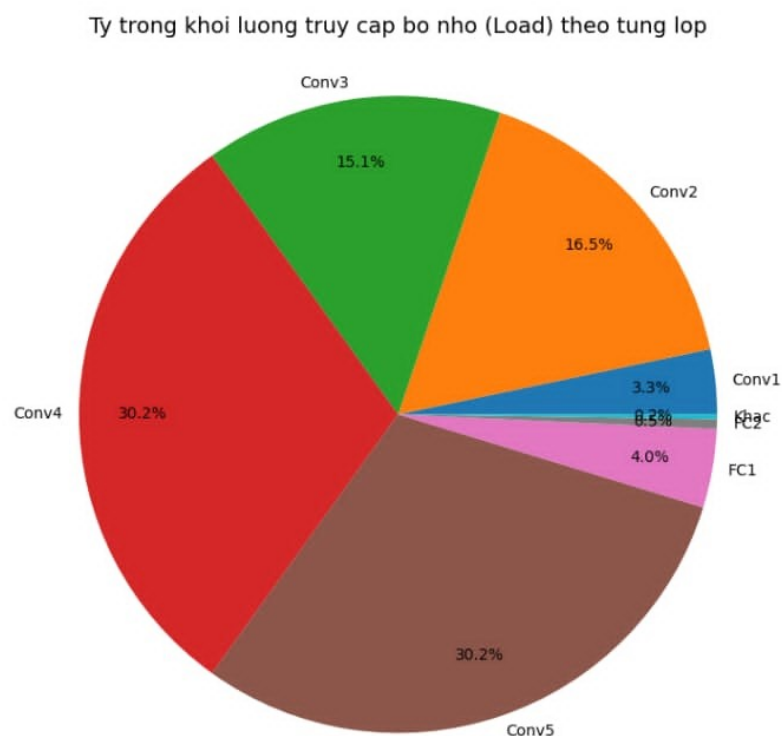
Hình 5: Ảnh hưởng của tile size đến thời gian suy luận toàn mạng

Có hai quan sát quan trọng. Thứ nhất, tổng số lần gọi load/compute/store không đổi giữa các kích thước tile — việc tiling ở đây chỉ thay đổi thứ tự duyệt các điểm ảnh đầu ra (oh, ow), không làm giảm khối lượng tính toán về mặt lý thuyết. Nói cách khác, phép tiling trong mô-đun 2 tối ưu tính cục bộ bộ nhớ cache (locality), không làm giảm khối lượng công việc lý thuyết — khác với mô-đun 1, nơi cách tổ chức Local Buffer thực sự làm giảm số lần truy cập DRAM.

Thứ hai, trên máy chủ dùng để chạy thử nghiệm (CPU đa lõi với cache L1/L2 khá lớn so với kích thước bài toán CIFAR-10 32×32), kích thước tile 1×1 (không tiling) lại cho thời gian thấp nhất, và thời gian tăng dần khi tile lớn hơn. Kết quả này vẫn phù hợp với phân tích lý thuyết ở Mục 2.3, mà phản ánh đúng bản chất của tiling: lợi ích chỉ xuất hiện khi kích thước bài toán vượt quá dung lượng cache sẵn có. Với bài toán CIFAR-10 kích thước nhỏ (tối đa $32 \times 32 \times 64$ phần tử một lớp), toàn bộ dữ liệu một lớp đã vừa cache L2 của CPU máy chủ, nên việc chia tile chỉ làm tăng chi phí quản lý vòng lặp mà không mang lại lợi ích cache. Trên hệ thống nhúng mục tiêu (vi điều khiển với SRAM on-chip nhỏ hơn nhiều, ví dụ vài chục đến vài trăm KB), dự kiến xu hướng sẽ đảo ngược: tile vừa với SRAM sẽ nhanh hơn baseline. Đây là hạn chế quan trọng cần lưu ý khi diễn giải kết quả mục 4.2 (xem thêm mục 4.4).

4.3 Phân tích tỉ trọng theo từng lớp mạng

Bảng breakdown chi tiết theo từng lớp (ứng với tile tốt nhất đo được, 1×1) như sau:



Hình 6: Tỉ trọng khối lượng truy cập bộ nhớ (Load) theo từng lớp

<i>Lớp</i>	<i>Load</i>	<i>Compute</i>	<i>Store</i>
<i>Conv1</i>	<i>1.729.280</i>	<i>1.696.512</i>	<i>32.768</i>
<i>ReLU1 / Pool1</i>	<i>65.536</i>	<i>65.536</i>	<i>40.960</i>
<i>Conv2</i>	<i>8.683.520</i>	<i>8.667.136</i>	<i>16.384</i>
<i>ReLU2 / Pool2</i>	<i>32.768</i>	<i>32.768</i>	<i>20.480</i>
<i>Conv3</i>	<i>7.938.048</i>	<i>7.929.856</i>	<i>8.192</i>
<i>Conv4</i>	<i>15.867.904</i>	<i>15.859.712</i>	<i>8.192</i>
<i>Conv5</i>	<i>15.867.904</i>	<i>15.859.712</i>	<i>8.192</i>
<i>ReLU5 / Pool5</i>	<i>16.384</i>	<i>16.384</i>	<i>10.240</i>
<i>FC1</i>	<i>2.097.664</i>	<i>2.097.152</i>	<i>512</i>
<i>FC2</i>	<i>262.400</i>	<i>262.144</i>	<i>256</i>
<i>FC3 / Softmax</i>	<i>5.159</i>	<i>5.159</i>	<i>30</i>
<i>TỔNG</i>	<i>52.582.951</i>	<i>52.508.455</i>	<i>162.590</i>

Bảng 4.5: Bảng tổng hợp kết quả tính toán chi tiết theo từng lớp

Ba lớp Conv2, Conv4 và Conv5 chiếm tổng cộng hơn 76% tổng khối lượng load-/compute của toàn mạng — phù hợp với nhận định rằng các lớp tích chập có số kênh vào–ra lớn ($64 \rightarrow 64$, $128 \rightarrow 128$) là nơi tập trung phần lớn tải tính toán. Đây chính là các lớp nên được ưu tiên tối ưu dataflow (chọn chiến lược OS/WS/IS phù hợp) và tiling khi triển khai thực tế, vì mức lợi ích thu được trên các lớp này sẽ đóng góp chủ yếu vào tổng thời gian suy luận.

4.4 Nhận xét và hạn chế

Mô-đun 1 (phân tích dataflow OS/WS/IS) cho kết quả đo đạc tin cậy, phản ánh đúng đặc tính lý thuyết của từng chiến lược và đều vượt qua bài kiểm tra đối chiếu sai số nghiêm ngặt với hàm nội tại của PyTorch.

Mô-đun 2 (khảo sát tile size toàn mạng) đã được triển khai thành công với bộ trọng số thực tế, được huấn luyện trực tiếp trên tập dữ liệu CIFAR-10 thông qua công cụ `verify_combined.py`. Chương trình C đã nạp thành công bộ trọng số được xuất từ PyTorch và thực hiện quá trình suy luận trên ảnh đầu vào. Kết quả thu được phù hợp với kết quả đối chiếu từ PyTorch trong các trường hợp thử nghiệm, cho thấy quá trình chuyển đổi mô hình từ Python sang C được hiện thực đúng theo thiết kế.

Tuy nhiên, nghiên cứu vẫn còn một số hạn chế nhất định. Kết quả đo đặc tốc độ theo kích thước tile hiện tại chịu sự ảnh hưởng từ kiến trúc bộ nhớ đệm (Cache) lớn của máy chủ mô phỏng. Để xác định kích thước tile phù hợp cho thiết bị nhúng (ví dụ vi điều khiển STM32H723ZG), cần thực hiện đo đặc trực tiếp trên phần cứng mục tiêu hoặc bổ sung mô hình bộ nhớ SRAM thực tế vào mã nguồn.

Bên cạnh đó, chương trình hiện tại đang sử dụng hàm `clock()` (độ phân giải mili-giây) để đánh giá thời gian thực thi, chưa mô hình hóa được mức tiêu thụ năng lượng — một tham số quan trọng không kém khi đánh giá dataflow cho các hệ thống nhúng hạn chế tài nguyên.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Đồ án đã xây dựng một hệ thống mô phỏng bằng ngôn ngữ C cho phép phân tích và so sánh ba chiến lược dataflow OS/WS/IS trên một lớp tích chập. Kết quả thực nghiệm cho thấy các đặc điểm của ba chiến lược dataflow phù hợp với phân tích lý thuyết: chiến lược OS tối ưu hóa số lần ghi dữ liệu, WS giảm tải nạp trọng số, và IS tiết kiệm băng thông nạp dữ liệu đầu vào. Kết quả tính toán đạt độ chính xác cao khi đối chiếu với PyTorch. Đồng thời, đề tài đã tích hợp bộ trọng số huấn luyện thực tế vào mạng TinyCNN bằng C, qua đó tiến hành khảo sát kỹ thuật Tiling và có thể giữ nguyên mối tương quan giữa kích thước khối dữ liệu (tile size) với đặc tính phân cấp bộ nhớ của phần cứng.

Từ những kết quả đạt được, nhóm đề xuất các hướng phát triển tiếp theo nhằm hoàn thiện và đưa giải pháp vào ứng dụng thực tiễn:

- Mô hình hóa Local Buffer toàn mạng: Thay vì chỉ áp dụng vòng lặp Tiling cho quá trình tính toán, cần giả lập dung lượng Local Buffer chi tiết cho Mô-đun 2 (tương tự cách đã làm ở Mô-đun 1). Điều này giúp đo lường chính xác lượng dữ liệu truyền tải qua DRAM được giảm thiểu khi tile size vừa vặn với kích thước SRAM mục tiêu.
- Tùy biến Dataflow theo lớp mạng: Dựa trên phân tích tải trọng ở mục 4.3, hệ thống nên áp dụng chiến lược OS, WS, hoặc IS riêng biệt cho từng lớp cụ thể của TinyCNN thay vì cố định một chiến lược xuyên suốt mạng, nhằm tối ưu hóa hiệu năng tổng thể.
- Kết hợp lượng tử hóa (Quantization): Áp dụng chuẩn dữ liệu số nguyên INT8 thay cho số thực (float32) kết hợp cùng tối ưu dataflow. Sự cộng hưởng của hai phương pháp này sẽ giúp giảm thiểu băng thông bộ nhớ và năng lượng tiêu thụ trên chip.
- Triển khai thực địa (Hardware Deployment): Đưa toàn bộ mã nguồn biên dịch và vận hành trực tiếp trên vi điều khiển thực tế (như bo mạch STM32H723ZG) để đo lường thông số phần cứng thật, qua đó so sánh độ sai lệch giữa môi trường mô phỏng và vi mạch thực tế.

TÀI LIỆU THAM KHẢO

- [1] Z. Li, F. Liu, W. Yang, S. Peng, and J. Zhou, "A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 12, pp. 4793–4823, Dec. 2021.
- [2] K. O'Shea and R. Nash, "An Introduction to Convolutional Neural Networks," *arXiv preprint arXiv:1511.08458*, 2015.
- [3] D. Li, X. Chen, M. Becchi, and Z. Zong, "Evaluating the Energy Efficiency of Deep Convolutional Neural Networks on CPUs and GPUs," in *Proceedings of the IEEE International Symposium on Embedded Computing Systems: Architectures, Modeling and Simulation (SAMOS)*, Jul. 2016, pp. 1–8.
- [4] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks," in *ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 367–379.
- [5] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.